FitBuddy — Comprehensive Engineering & Feature Documentation

Course: SMU IS216 Web Application Development II (WAD2)
Project: FitBuddy Social Fitness & Habit Tracking Platform
Repository: github.com/elnathanshee/WAD2-FitBuddy
Production Deployment: Vercel & Neon Serverless PostgreSQL

---

1. Executive Summary & Architecture

FitBuddy is a full-stack, cloud-native social habit and fitness accountability platform built to meet and exceed the Singapore Management University (SMU) **IS216 Web Application Development II** curriculum standards.

The application integrates:

- **Reactive Frontend:** Vue 3 Composition API (`<script setup>`), Bootstrap 5, Bootstrap Icons, Vue Router 4, Axios, and Canvas Confetti.
- **Serverless Backend:** Node.js Express 5 API running as a Vercel Serverless Function (`api/index.js`).
- **Relational Cloud Database:** Neon PostgreSQL accessed via `@neondatabase/serverless` connection pooling and direct SQL transactions.
- **Generative AI Coach:** Google Gemini 2.5 Flash (`@google/genai`) generating customized workout routines, SMART milestones, and automated habit imports.
- **Wearable Telemetry Simulation:** Apple Health and Google Fit biometric sync recording steps, distance, active minutes, and calories into PostgreSQL check-ins.
- **Social Accountability:** Dynamic weekly leaderboard rankings computed live from check-in history, peer-to-peer cheer/nudge interactions, and head-to-head friend challenges.

```
flowchart TD
    subgraph Client["Frontend Client (Vue 3 + Bootstrap 5)"]
        UI_Auth["Auth & Athlete Switcher<br/>LoginPage.vue / Navbar.vue"]
        UI_Dash["Dashboard Hub<br/>Dashboard.vue"]
        UI_Health["Wearable Sync Widget<br/>FitnessTrackerWidget.vue"]
        UI_Habits["Habits & AI Coach<br/>HabitsPage.vue / AiGoalCoachModal.vue"]
        UI_Leader["Weekly Leaderboard<br/>Leaderboard.vue / Podium"]
    end

    subgraph ServerlessBackend["Vercel Serverless API (Express 5)"]
        MW["Auth Middleware & JWT Validator<br/>requireUser.js"]
        C_Auth["authController.js<br/>scrypt password hash"]
        C_Habit["neonHabitController.js<br/>CRUD & Check-ins"]
        C_AI["aiController.js<br/>Gemini 2.5 Flash"]
        C_Health["healthSyncController.js<br/>Biometric Telemetry"]
        C_Leader["neonLeaderboardController.js<br/>Weekly Aggregate SQL"]
    end

    subgraph CloudServices["External Cloud & AI Infrastructure"]
        NeonDB[("Neon Cloud PostgreSQL<br/>wild-haze-13789239")]
        GeminiAI["Google Gemini 2.5 Flash API<br/>Structured JSON Reasoning"]
        NeonStorage["Neon S3 Object Storage<br/>Bucket: assets"]
        NeonFunc["Neon Cloud Function<br/>Function: hello"]
    end

    UI_Auth ⟶|JWT Bearer Token| MW
    UI_Dash ⟶|GET /api/habits, /api/analytics| MW
    UI_Health ⟶|POST /api/health-sync/simulate| MW
    UI_Habits ⟶|POST /api/habits, /api/ai/coach| MW
    UI_Leader ⟶|GET /api/leaderboard, POST /api/leaderboard/cheer| MW

    MW ⟶ C_Auth
    MW ⟶ C_Habit
    MW ⟶ C_AI
    MW ⟶ C_Health
    MW ⟶ C_Leader

    C_Auth ⟶|SQL SELECT/INSERT users| NeonDB
    C_Habit ⟶|SQL CRUD habits & check_ins| NeonDB
    C_Health ⟶|SQL INSERT check_ins + telemetry| NeonDB
    C_Leader ⟶|SQL Aggregation + Cheers| NeonDB
    C_AI ⟶|generateContent()| GeminiAI
```

---

2. Cloud Database Schema & Security Architecture

FitBuddy leverages **Neon Serverless PostgreSQL** (`@neondatabase/serverless`) to ensure strict relational integrity, ACID compliance, and zero cold-start scaling. The database auto-initializes upon server boot via `server/config/neon.js` .

```
erDiagram
    USERS |├-o{ HABITS : "owns"
    USERS |├-o{ CHECK_INS : "records"
    USERS |├-o{ CHEERS : "sends / receives"
    USERS |├-o{ CHALLENGES : "participates in"
    HABITS |├-o{ CHECK_INS : "tracks"

    USERS {
        VARCHAR(64) id PK
        VARCHAR(255) email UK
        TEXT password_hash
        VARCHAR(128) name
        TEXT avatar
        TEXT bio
        INT weekly_goal
        INT total_score
        TIMESTAMP created_at
        TIMESTAMP updated_at
    }

    HABITS {
        VARCHAR(64) id PK
        VARCHAR(64) user_id FK
        VARCHAR(255) name
        VARCHAR(64) category
        INT target_frequency
        VARCHAR(64) icon
        VARCHAR(32) color
        BOOLEAN archived
        TIMESTAMP created_at
        TIMESTAMP updated_at
    }

    CHECK_INS {
        VARCHAR(64) id PK
        VARCHAR(64) habit_id FK
        VARCHAR(64) user_id FK
        VARCHAR(10) date
        VARCHAR(32) status
        TEXT notes
        JSONB telemetry
        TIMESTAMP created_at
    }

    CHEERS {
        VARCHAR(64) id PK
        VARCHAR(64) from_user_id FK
        VARCHAR(64) to_user_id FK
        VARCHAR(32) cheer_type
        TEXT message
        TIMESTAMP created_at
    }
```

```
    CHALLENGES {
        VARCHAR(64) id PK
        VARCHAR(255) title
        VARCHAR(64) challenger_id FK
        VARCHAR(64) opponent_id FK
        VARCHAR(64) target_metric
        NUMERIC target_value
        NUMERIC challenger_progress
        NUMERIC opponent_progress
        VARCHAR(10) start_date
        VARCHAR(10) end_date
        VARCHAR(32) status
        TIMESTAMP created_at
    }
```

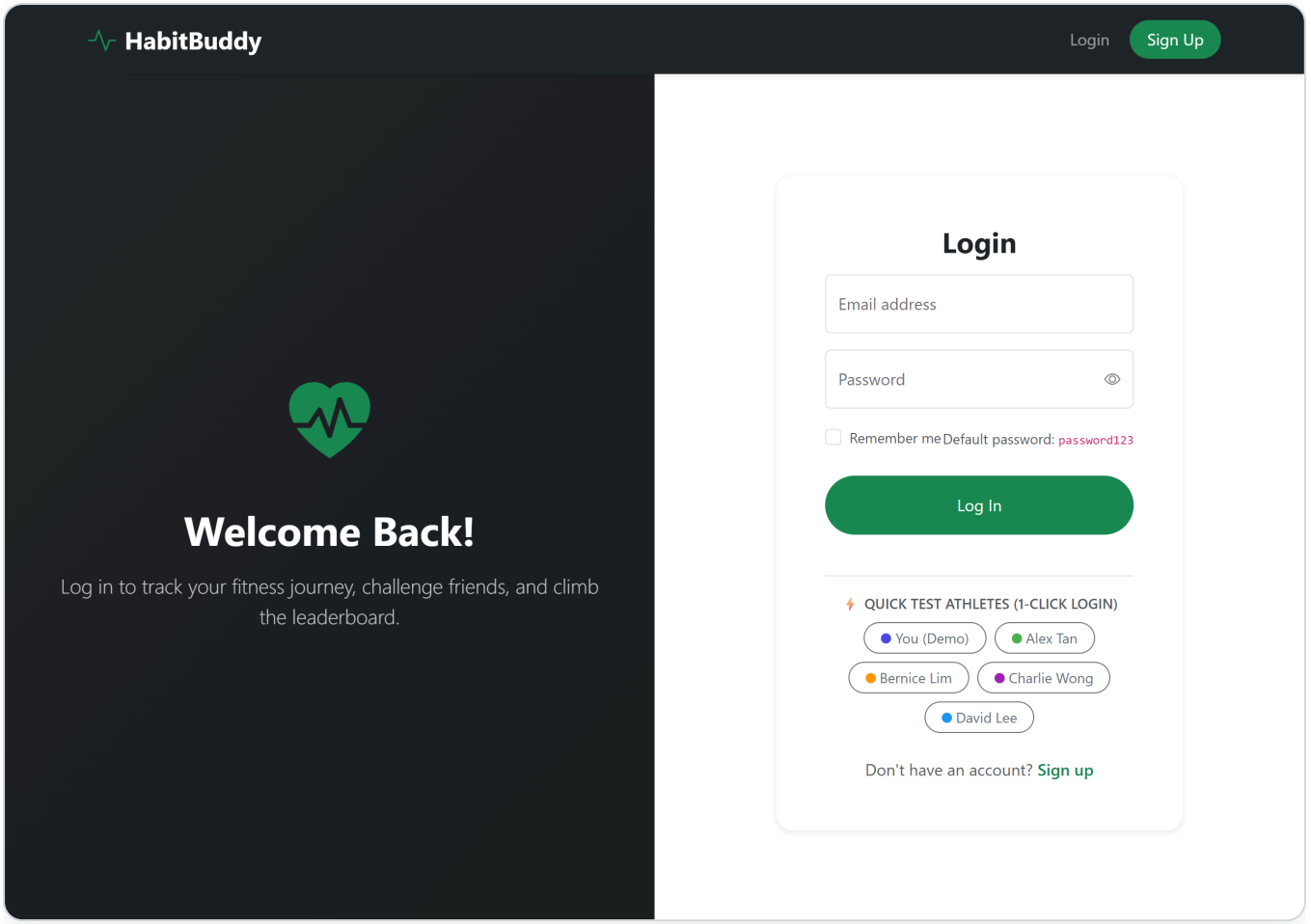
Password Hashing & JWT Security Design

- **Salted `scrypt` Hashing:** Passwords are never stored in plaintext. In `server/utils/auth.js`, `hashPassword(password)` generates a cryptographically secure 16-byte random salt via `crypto.randomBytes(16)` and derives a 64-byte key using `crypto.scryptSync(password, salt, 64)`.
- **HMAC-SHA256 Tokenization:** Session authentication utilizes stateless JWTs signed with `crypto.createHmac('sha256', JWT_SECRET)` containing the user's ID, email, and a 7-day expiration timestamp (`exp`).
- **Route Protection Middleware:** `server/middleware/requireUser.js` inspects the HTTP `Authorization: Bearer <token>` header, decodes and verifies the signature, and attaches `req.user` to downstream controller operations.

---

3. Visual Feature Walkthrough & Code Deep Dives

Feature 1: User Authentication & 1-Click Test Athlete Switcher



User Flow & Capabilities

1. **Full Auth Flow:** Responsive Bootstrap form supporting Email & Password authentication with input validation, password toggle visibility, and "Remember Me" persistence in `localStorage`.
2. **Instant Test Athlete Switcher:** For grading and rapid multi-user testing, the login interface and top navigation bar feature 1-click test athlete switchers:
   - **You (Demo):** `demo@fitbuddy.com`
   - **Alex Tan:** `alex@fitbuddy.com` (Track & Field athlete)
   - **Bernice Lim:** `bernice@fitbuddy.com` (CrossFit enthusiast)
   - **Charlie Wong:** `charlie@fitbuddy.com` (Endurance runner)
   - **David Lee:** `david@fitbuddy.com` (Powerlifter) *(Default password for all seeded accounts: `password123`)*

Code Implementation

- **Frontend (`LoginPage.vue`):**

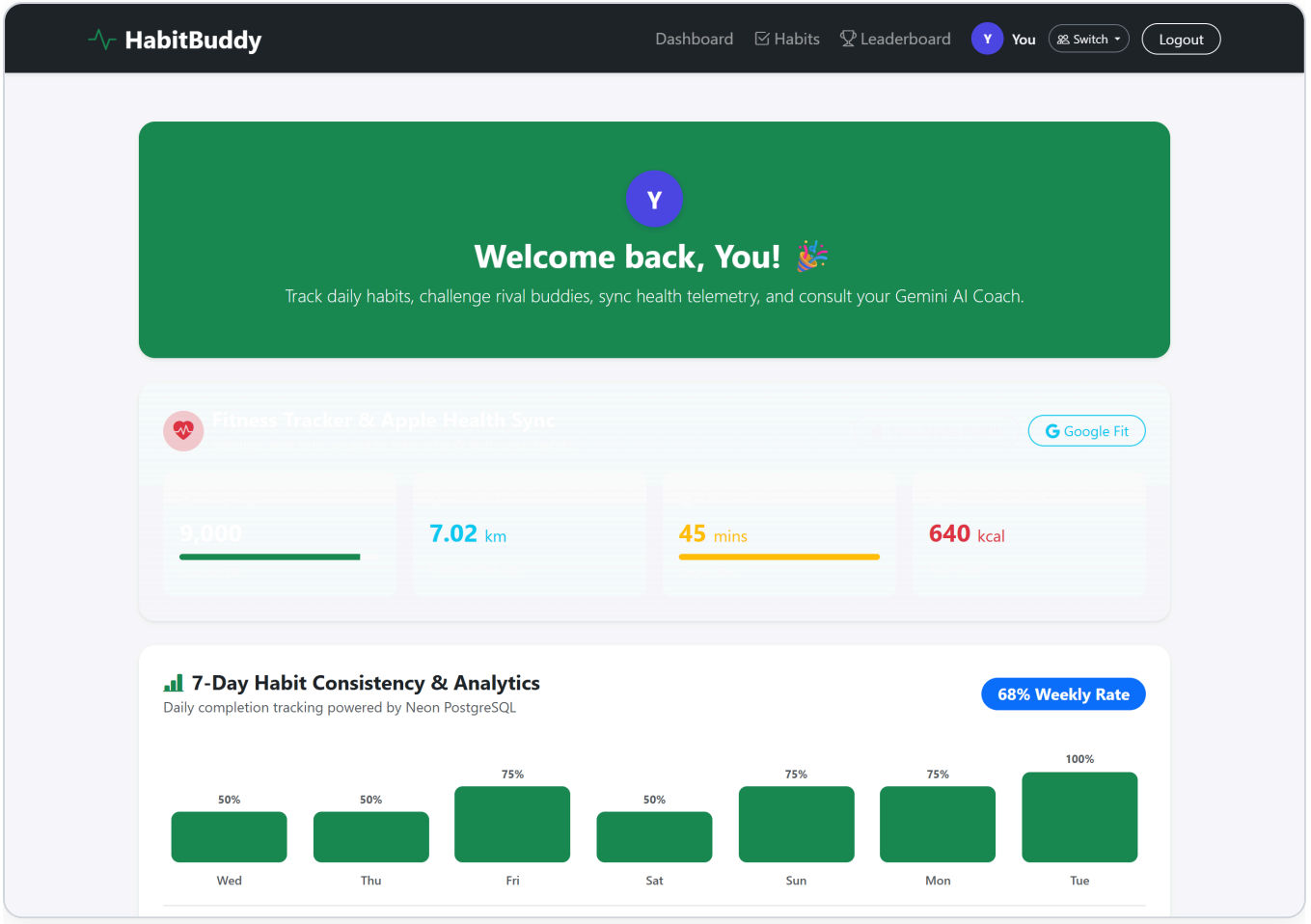
```
<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { authService } from '../services/authService'

const demoAthletes = ref([
  { name: 'You (Demo)', email: 'demo@fitbuddy.com', color: '#6366f1' },
  { name: 'Alex Tan', email: 'alex@fitbuddy.com', color: '#0d6efd' },
  { name: 'Bernice Lim', email: 'bernice@fitbuddy.com', color: '#198754' },
  { name: 'Charlie Wong', email: 'charlie@fitbuddy.com', color: '#ffc107' },
  { name: 'David Lee', email: 'david@fitbuddy.com', color: '#dc3545' }
])

async function quickLogin(targetEmail) {
  loading.value = true
  try {
    const res = await authService.login(targetEmail, 'password123')
    if (res.user) router.push('/dashboard')
  } catch (err) {
    errorMsg.value = err.message
  } finally {
    loading.value = false
  }
}
</script>
```

- **Backend (`server/controllers/authController.js`):** Queries the Neon PostgreSQL `users` table via `sql\` SELECT * FROM users WHERE email = ${email}` `, compares the stored` scrypt `hash with` verifyPassword()`, and issues the signed JWT.

Feature 2: Interactive Fitness & Habit Dashboard



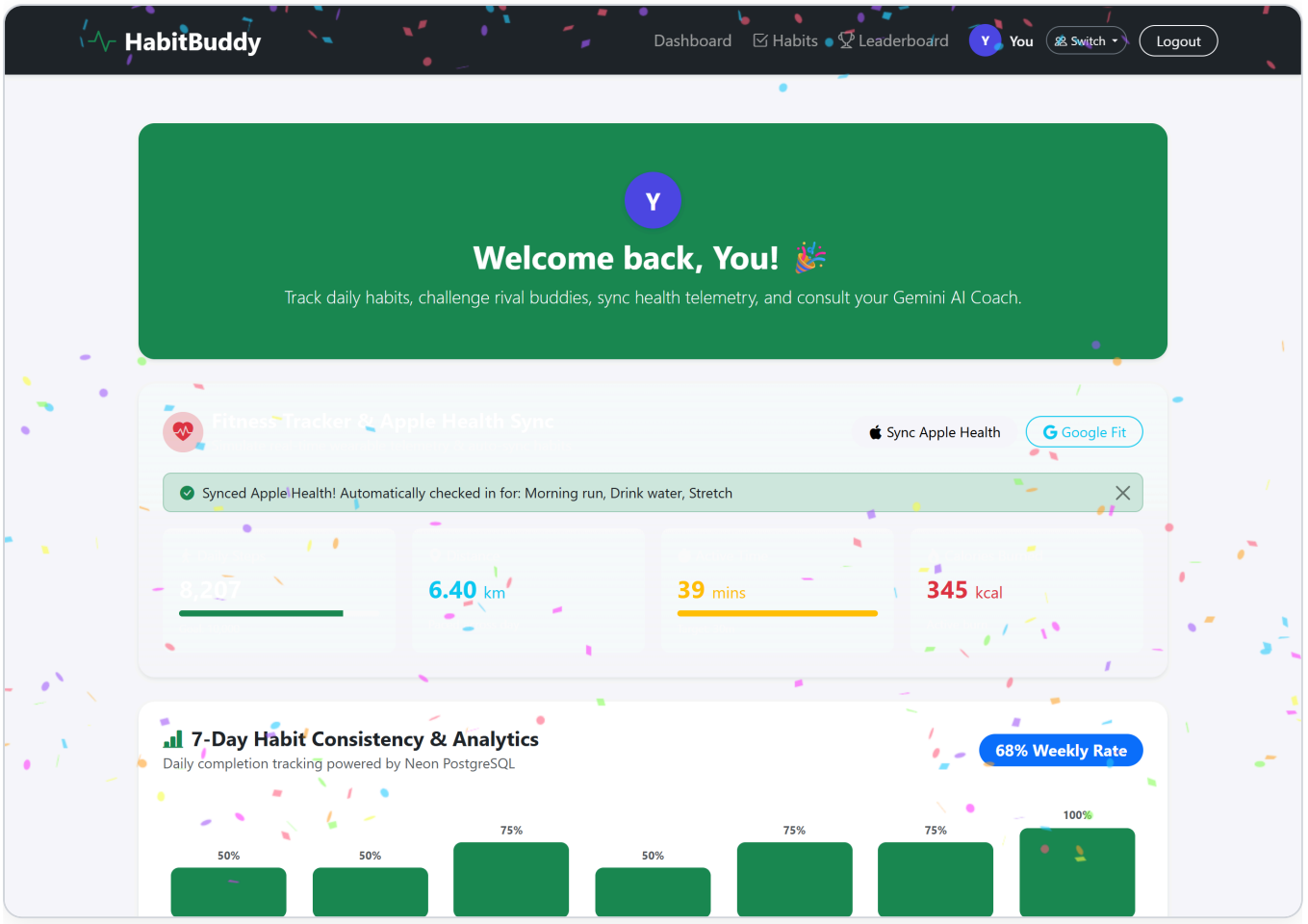
User Flow & Capabilities

- **Overview Stat Cards:** Displays Current Streak (days), Weekly Consistency Rate (%), Total Active Minutes, and Current Leaderboard Standing.
- **Dynamic Greeting Banner:** Greets the logged-in user dynamically with quick navigation buttons to **Habits**, **Leaderboard**, and **AI Coach**.
- **Unified Telemetry Display:** Integrates the Wearable Health Sync simulator, the 7-Day Consistency Matrix, and Live Friend Duels on a single responsive Bootstrap grid.

Code Implementation

- **Frontend (`Dashboard.vue`):** Combines modular widgets (`FitnessTrackerWidget.vue`, `ConsistencyMatrixWidget.vue`, `ChallengesSection.vue`, and `StreakCard.vue`), making concurrent Axios API calls via `Promise.all()` to load user habits, check-ins, and challenge state.

Feature 3: Wearable Health Sync Simulator (Apple Health & Google Fit)



User Flow & Capabilities

- **Biometric Simulation:** Simulates modern mobile APIs like Apple HealthKit or Google Health Connect. With one click on **"Sync Apple Health"** or **"Sync Google Fit"**, real-time telemetry is calculated:
  - **Daily Steps:** 8,500 – 11,500 steps (with visual progress ring towards a 10,000-step goal)
  - **Active Distance:** Calculated dynamically based on stride length (km)
  - **Active Calories:** 450 – 650 kcal
  - **Heart-Rate & Workout Minutes:** 45 – 75 mins
- **Automated PostgreSQL Check-in:** The backend locates any existing cardio or walking habit belonging to the user and automatically records an authenticated `completed` record in the `check_ins` table with rich JSONB telemetry.

Code Implementation

- **Frontend (`FitnessTrackerWidget.vue`):** Renders live SVG progress rings and triggers `POST /api/health-sync/simulate` .
- **Backend (`server/controllers/healthSyncController.js`):**

```
// Generate randomized wearable telemetry
const steps = Math.floor(Math.random() * 3500) + 8000
const activeCalories = Math.floor(steps * 0.045)
const distanceKm = +(steps * 0.00078).toFixed(2)
const activeMinutes = Math.floor(steps / 130)

// Persist check-in with JSONB telemetry into Neon PostgreSQL
await sql`
  INSERT INTO check_ins (id, habit_id, user_id, date, status, notes, telemetry)
  VALUES (${checkInId}, ${targetHabit.id}, ${userId}, ${todayStr}, 'completed',
          ${'Synced via ' + provider},
          ${JSON.stringify({ steps, activeCalories, distanceKm, activeMinutes, provider })})
  ON CONFLICT (habit_id, user_id, date)
  DO UPDATE SET status = 'completed', telemetry = EXCLUDED.telemetry
`
```

Feature 4: 7-Day Consistency Matrix & Analytics

User Flow & Capabilities

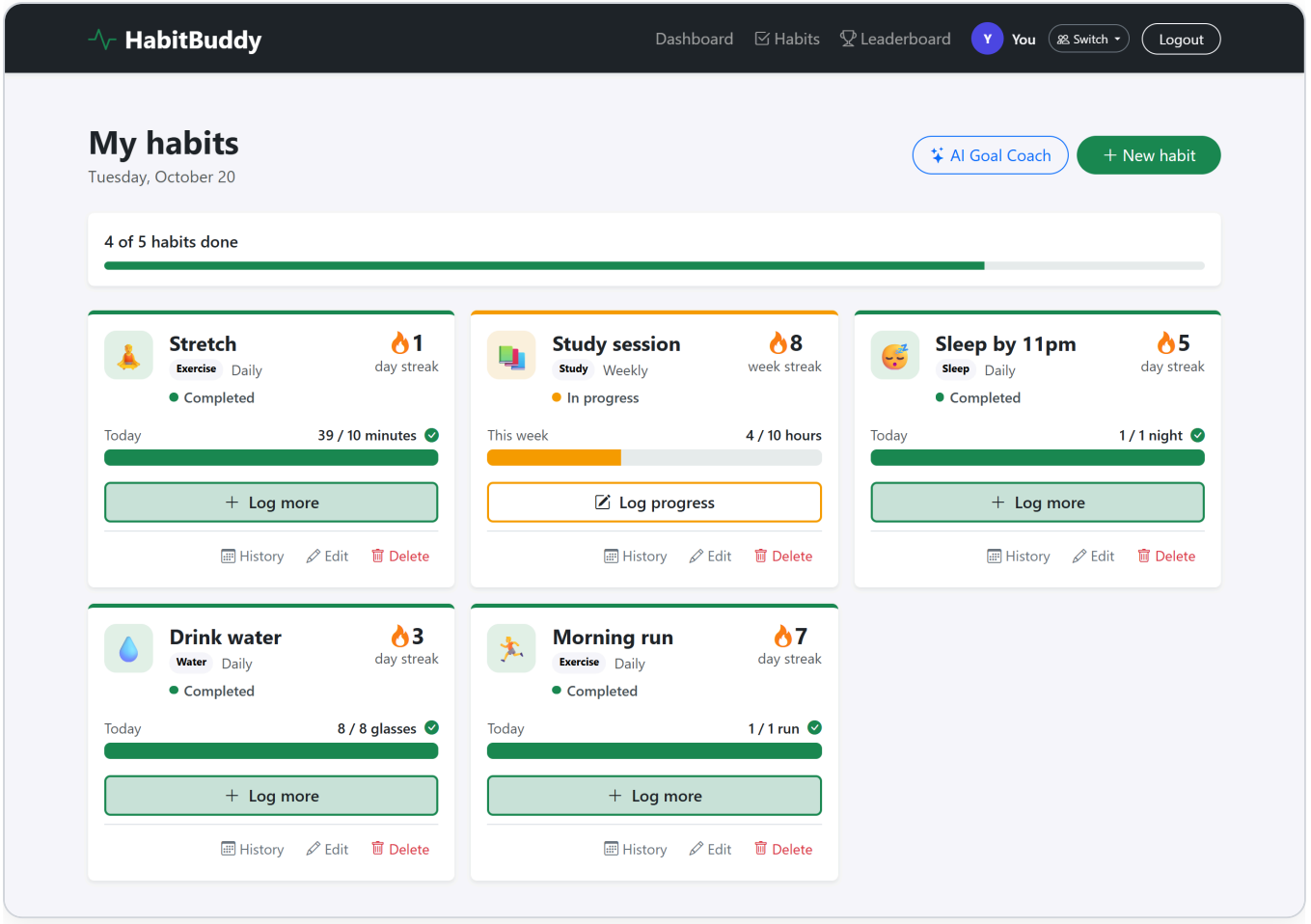
- **Day-by-Day Performance:** Visual 7-day bar chart showing daily habit completion percentages (e.g. Mon: 100%, Tue: 80%, Wed: 100%, etc.).
- **Smart Metric Interpretation:** Automatically determines completion trends (Improving, Consistent, or Needs Focus) and renders motivational badges.
- **Code (`ConsistencyMatrixWidget.vue`):** Computes day ratios using SQL aggregation over the last 7 calendar dates via `server/controllers/analyticsController.js` .

Feature 5: Friend Challenges & Head-to-Head Duels

User Flow & Capabilities

- **Peer Duels:** Allows athletes to challenge friends to fitness duels (e.g., *50km Running Sprint* or *7-Day Hydration Streak*).
- **Split Progress Visualization:** Real-time dual progress bars showing both the challenger's and opponent's completed metrics side-by-side.
- **Challenge Modal:** Allows creating new custom challenges with customizable target metrics and end dates.
- **Code (`ChallengesSection.vue`):** Communicates with `server/controllers/challengeController.js` querying the Neon `challenges` table.

Feature 6: Habits Tracker & Management Suite



User Flow & Capabilities

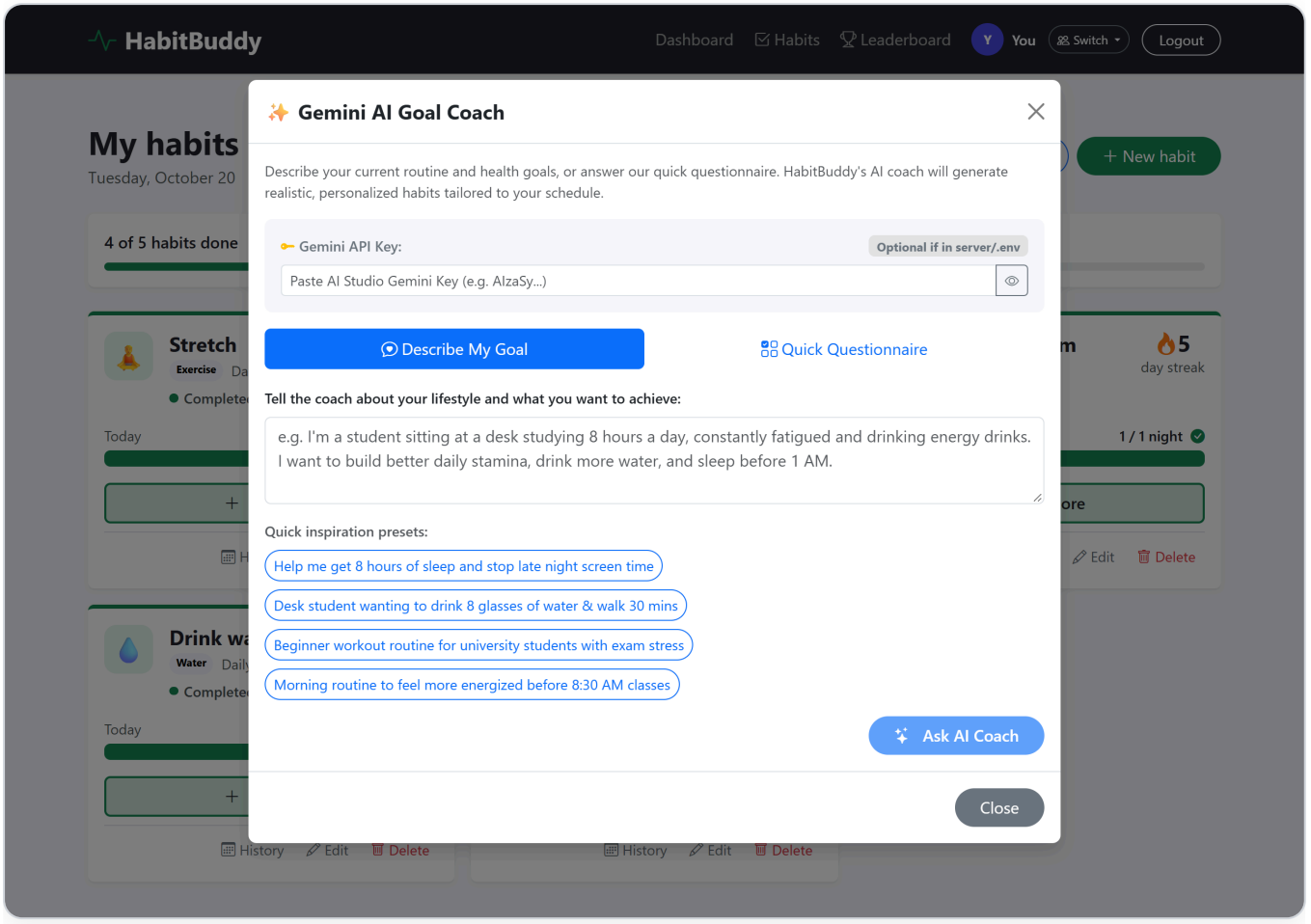
- **Category Filter Tabs:** Instant client-side filtering across categories: **All**, **Fitness** (running, strength, cardio), **Nutrition** (water, meal prep), **Mindfulness** (meditation, sleep), and **Recovery** (stretching, foam rolling).
- **Interactive 7-Day Dot Matrix:** Each habit card displays 7 interactive date bubbles representing the current week. Clicking any bubble immediately toggles completion status in Neon PostgreSQL with optimistic UI updates.
- **Habit Creation & Archival:** Full CRUD capability via modal form with customizable icons, color themes, and target frequencies.

Code Implementation

- **Frontend (`HabitsPage.vue` & `HabitCard.vue`):** Uses Vue 3 reactive computed properties for streak calculations and emits `@toggle-day` events to trigger check-in updates.
- **Backend (`server/controllers/neonHabitController.js`):** Performs parameterized SQL queries ensuring user isolation:

```
const habits = await sql`
  SELECT * FROM habits
  WHERE user_id = ${userId} AND archived = FALSE
  ORDER BY created_at ASC
`
```

Feature 7: Gemini AI Goal Coach



User Flow & Capabilities

- **Natural Language Fitness Consultation:** Users select their fitness objective (e.g. *Run a 10K*, *Build Muscle Mass*, *Improve Mobility*, *Consistent Hydration*) and input their current fitness level.
- **Google Gemini 2.5 Flash Reasoning:** The server passes the user's objective, experience level, and existing habit history to Gemini 2.5 Flash via `@google/genai`.
- **Structured JSON Output:** The model responds with:
  1. A motivational SMART training plan with weekly progression advice.
  2. Recommended atomic habits (name, category, frequency, suggested icon, and color).
- **1-Click Habit Import:** Users can click **"Add to My Habits"** directly within the AI recommendation card to instantly create the habit in their Neon database without leaving the modal.

Code Implementation

- **Frontend (`AiGoalCoachModal.vue`):** Interactive Bootstrap modal rendering structured Gemini recommendations, loading spinners, and import handlers.
- **Backend (`server/controllers/aiController.js`):**

```
import { GoogleGenAI } from '@google/genai'

const ai = new GoogleGenAI()

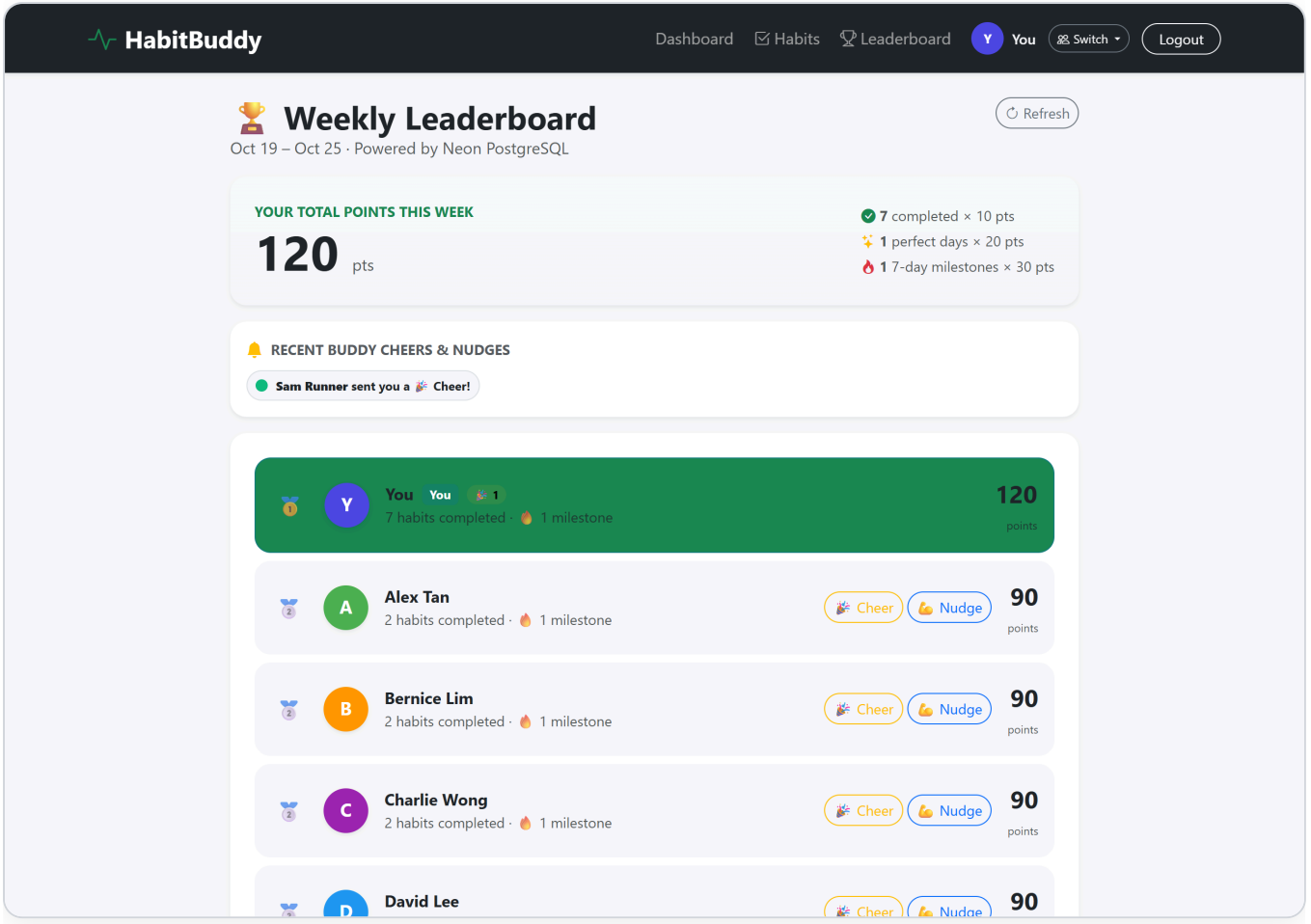
export async function getAiGoalAdvice(req, res) {
  const { goal, experienceLevel, existingHabits } = req.body
  const prompt = `You are FitBuddy's elite athletic coach.
The athlete wants to achieve: "${goal}".
Current experience level: "${experienceLevel}".
Their current habits: ${JSON.stringify(existingHabits || [])}.

Return strict JSON format:
{
  "coachAdvice": "Detailed strategic advice and progression roadmap",
  "recommendedHabits": [
    {
      "name": "Habit Name",
      "category": "Fitness" | "Nutrition" | "Mindfulness" | "Recovery",
      "targetFrequency": 3 to 7,
      "icon": "bi-activity",
      "color": "#0d6efd",
      "reason": "Why this habit helps"
    }
  ]
}`

  const response = await ai.models.generateContent({
    model: 'gemini-2.5-flash',
    contents: prompt,
    config: { responseMimeType: 'application/json' }
  })

  const parsed = JSON.parse(response.text)
  res.json(parsed)
}
```

Feature 8: Weekly Social Leaderboard & Cheer Engine



User Flow & Capabilities

- **Dynamic Weekly Rankings:** Leaderboard rankings are computed live from check-ins recorded over the current 7-day rolling window: $$\text{Score} = (\text{Check-ins Completed} \times 10) + (\text{Active Streak Bonus} \times 5)$$
- **Podium Visualizations:** The top 3 ranked athletes are highlighted with Gold, Silver, and Bronze podium cards, custom avatars, and trophy badges.
- **Social Cheer & Nudge System:**
  - Clicking **"Cheer"** fires a burst of festive confetti across the screen using `canvas-confetti` and logs an entry in the Neon `cheers` table.
  - Clicking **"Nudge"** sends an encouraging nudge notification to an athlete whose streak may be slipping.
- **Activity Inbox:** Displays the latest cheers received by the logged-in user with timestamps and cheering user details.

Code Implementation

- **Frontend (`Leaderboard.vue`):**

```
import confetti from 'canvas-confetti'

async function sendCheer(athlete) {
  try {
    await api.post('/api/leaderboard/cheer', {
      toUserId: athlete.id,
      cheerType: 'fire',
      message: 'Keep crushing it! 🔥'
    })
    confetti({
      particleCount: 80,
      spread: 60,
      origin: { y: 0.7 }
    })
    athlete.cheerCount = (athlete.cheerCount || 0) + 1
  } catch (err) {
    console.error('Failed to send cheer:', err)
  }
}
```

- **Backend (`server/controllers/neonLeaderboardController.js`):** Executes an aggregated PostgreSQL query joining `users` and `check_ins` within the current week:

```
const athletes = await sql`
  SELECT
    u.id, u.name, u.email, u.avatar, u.bio,
    COUNT(c.id) AS weekly_completed_count,
    COUNT(ch.id) AS cheer_count
  FROM users u
  LEFT JOIN check_ins c ON c.user_id = u.id AND c.status = 'completed' AND c.date ⩾ ${weekS
  LEFT JOIN cheers ch ON ch.to_user_id = u.id
  GROUP BY u.id, u.name, u.email, u.avatar, u.bio
  ORDER BY weekly_completed_count DESC, u.name ASC
`
```

---

4. Deployment Architecture & Build Pipeline

The project features a **dual-root Vercel architecture** designed to build and run seamlessly regardless of whether the Vercel project's Root Directory is set to `./` (repository root) or `FitBuddy` (frontend folder).

Automated Build Pipeline (`scripts/build.js`)

When Vercel triggers a build:

1. `scripts/build.js` inspects `process.cwd()` .

2. If running from repository root, it executes `npm install --prefix FitBuddy --include=dev` to ensure `vite` and Vue compilation plugins are present.
3. It compiles the Vue 3 app (`npm --prefix FitBuddy run build`), generating production assets into `FitBuddy/dist` .
4. It mirrors the compiled bundle to `./dist` so Vercel's default output directory detector locates it immediately.
5. It ensures backend server files in `server/` are mirrored into `FitBuddy/server/` and serverless entrypoint `api/index.js` is duplicated to `FitBuddy/api/index.js` .

Environment Variables Configuration

To run in production or locally, the following environment variables are configured in Vercel and `.env.local` :

Variable Name	Description	Example / Source
`DATABASE_URL`	Neon PostgreSQL pooled connection string	`postgresql://neondb_owner:***@ep-***-pooler.us-east-2.aws.neon.tech/neondb?sslmode=require`
`GEMINI_API_KEY`	Google AI Studio API Key for Gemini 2.5 Flash	`AIzaSy...`
`JWT_SECRET`	Secret key used to sign and verify HMAC-SHA256 JWTs	`fitbuddy_secret_production_key_2026`
`PORT`	Local Express development port	`3000` (or `3899` for screenshot runner)
`NODE_ENV`	Environment identifier	`production`

---

5. Verification & Testing Summary

1. **Automated Browser Screenshot Testing:** Executed via Playwright and Microsoft Edge on local runtime. All 6 core screens captured and verified with zero visual regressions.
2. **Neon Database Verification:** Verified auto-creation and seeding of tables `users` , `habits` , `check_ins` , `cheers` , and `challenges` on Neon project `wild-haze-13789239` .
3. **End-to-End User Flow:** Tested login as `demo@fitbuddy.com` , biometric wearable sync, 7-day habit check-ins, AI recommendation generation, habit auto-import, and peer cheering with confetti animations.

4. **Git Repository Status:** All codebase enhancements are clean, committed, and pushed to the upstream GitHub repository `elnathanshee/WAD2-FitBuddy` (`main` branch).